

Bilan du Sprint 2

Pipeline d'indexation complet – Système RAG pour hcp.ma

Structure d'accueil : Haut-Commissariat au Plan (HCP), Direction des Systèmes
d'Information Statistiques

Binôme : Ayman El Baida et Saad Mesbah

Période couverte : 14 – 20 juillet 2026 (Sprint 2)

Document rédigé le 20 juillet 2026 – support de suivi pour l'encadrante et le rapport de stage

Table des matières

1	Objectif du Sprint 2	2
2	Vue d'ensemble du pipeline	2
3	Découverte automatique des publications (ADR 0006)	2
3.1	Problème	2
3.2	Décision	2
3.3	Validation en conditions réelles	3
4	Support des fichiers DOCX (ADR 0005)	3
5	Chunking et embeddings	3
5.1	Chunking	3
5.2	Embeddings (BGE-M3)	4
6	Indexation hybride : Chroma (sémantique) + BM25 (lexical)	4
7	Persistance SQLite	5
8	Scripts d'orchestration	5
9	Tests automatisés	5
10	Validation en conditions réelles	6
10.1	Premier test, 3 documents	6
10.2	Vérification de la recherche hybride	6
10.3	Run élargi, 15 documents	7
10.4	Run complet, sans limite : 151 documents, DOCX validé	7
11	Limites connues et travail restant	7
12	Transition vers le Sprint 3	8

1 Objectif du Sprint 2

Le Sprint 1 avait établi que le RAG s'appuie exclusivement sur les pièces jointes PDF/XLSX/DOCX des publications hcp.ma (ADR 0003), avec un premier scraper capable de les télécharger sur un échantillon fixe de 27 URLs choisies à la main. Le Sprint 2 (14 – 20 juillet 2026) visait à transformer ces publications collectées en données réellement interrogeables : découpage du texte en fragments (*chunks*), calcul d'embeddings, indexation hybride sémantique + lexicale, et persistance en base SQLite – ainsi qu'à lever une limite identifiée en cours de sprint : la liste fixe de 27 URLs ne pouvait pas suivre les nouvelles publications de hcp.ma dans le temps.

Cinq blocs de travail ont été réalisés : (1) découverte automatique des publications, (2) support des fichiers DOCX, (3) chunking et embeddings, (4) indexation hybride Chroma + BM25, (5) persistance SQLite et scripts d'orchestration. Chacun est détaillé ci-dessous avec le code réel, puis validé par une exécution en conditions réelles sur la machine d'Ayman (section 8).

2 Vue d'ensemble du pipeline

Étape	Rôle
1. Scraper	Trouve les publications sur hcp.ma (découverte automatique, ADR 0006) et télécharge leurs pièces jointes (PDF/XLSX/DOCX).
2. Extracteur	Extrait le texte brut et les tableaux de chaque pièce jointe.
3. IndexeurTexte	Découpe le texte en chunks, calcule un embedding par chunk (BGE-M3), indexe dans Chroma (vectoriel) et BM25 (lexical).
4. base_donnees	Insère document, chunks et indicateurs en base SQLite, avec dédoublonnage.
5. Recherche hybride	À partir d'une question, fusionne un classement sémantique (Chroma) et lexical (BM25) par <i>Reciprocal Rank Fusion</i> .

3 Découverte automatique des publications (ADR 0006)

3.1 Problème

La liste fixe de 27 URLs du Sprint 1 ne peut ni couvrir l'historique complet des publications de hcp.ma, ni détecter automatiquement une nouvelle publication. Vérification faite sur le vrai site (`web_fetch`) : chaque sous-thème dispose d'une page "listing" paginée (ex. `Publications-Marche-du-travail_r425.html`) qui liste ses publications, les plus récentes en premier, avec une pagination réelle via `?start=N`.

3.2 Décision

`Scraper.decouvrir_urls_liste` lit une page listing, en extrait les URLs d'articles réels (filtrées par motif `_aNNNN.html` et présence dans un titre `<h2>–<h5>`, pour éviter les faux positifs de menu), puis suit la pagination. Le paramètre `max_pages` distingue deux usages : `max_pages=1` (page la plus récente seulement, exécution régulière de détection de fraîcheur) et `max_pages=None` (toutes les pages, collecte historique complète).

Listing 1: `src/scraper.py` – suivi de la pagination réelle

```
def decouvrir_urls_liste(self, url_listing, max_pages=None) -> list[str]:
    resp = requests.get(url_listing, headers=HEADERS, timeout=15)
    soup = BeautifulSoup(resp.text, "lxml")

    urls_trouvees = []
    for url in self._filtrer_urls_arabe(
        self._extraire_urls_articles(soup, url_listing)
    ):
        urls_trouvees.append(url)

    paliers = self._increments_pagination(soup)
```

```

if max_pages is not None:
    paliers = paliers[: max(0, max_pages - 1)]

for palier in paliers:
    url_page = f"{url_listing}?start={palier}&show=&order="
    resp = requests.get(url_page, headers=HEADERS, timeout=15)
    nouvelles = [...] # extraction + filtre langue, comme ci-dessus
    if not nouvelles:
        break # fin du listing atteinte
    urls_trouvees += nouvelles
return urls_trouvees

```

Le dédoublement entre deux exécutions successives ne se fait pas ici, mais repose sur la contrainte `document.url` UNIQUE déjà en place en base (voir section 7) : relancer la découverte ne recrée jamais un document déjà connu.

3.3 Validation en conditions réelles

La découverte a été exécutée pour de vrai (`scripts/decouverte_publications.py`) sur une page listing réelle : 10/10 URLs d'articles retrouvées, identiques à une vérification manuelle indépendante de la même page. Un problème réel est apparu à cette occasion : une page en version arabe (`...-version-Ar_a4217.html`) s'est glissée dans les résultats. Corrigé en réutilisant l'heuristique de détection de langue déjà validée en Sprint 1 pour les pièces jointes (`_filtrer_urls_arabe`), testé avec un cas reconstruit à partir de cet exemple réel.

L'inventaire des pages listing a été complété en parcourant réellement le site (`data/listing_urls.py`) : Population & démographie 10/10 sous-thèmes, Marché du travail 1 page agrégée (cas particulier confirmé), Économie partiellement (4 pages couvrant Comptes nationaux, Conjoncture, Conjoncture entreprise, Études économiques – reste à vérifier Indices des prix, Secteurs d'activité, Sphère informelle).

4 Support des fichiers DOCX (ADR 0005)

Découvert en Sprint 1 en testant l'extraction sur de vraies publications : certaines notes hcp.ma (indices IPC/IPPI notamment) sont diffusées en `.docx` plutôt qu'en PDF/XLSX. Décision prise avec Ayman le 17 juillet : étendre `Extracteur` plutôt que d'ignorer ce format, via `python-docx`. La détection du type de fichier réel (`_contenu_sembre_valide`) a dû être renforcée car DOCX et XLSX sont tous deux des archives ZIP au niveau binaire – distinction faite sur la présence de fichiers internes propres à chaque format (`word/document.xml` vs `xl/workbook.xml`).

5 Chunking et embeddings

5.1 Chunking

Aucune taille de chunk ni chevauchement n'était fixé dans les documents de conception (vérifié en relisant `fiche_cadrage_v4.pdf` §8.1 et `conception_u1_v3.pdf` §8, qui demandent un découpage "respectant la structure des documents" sans plus de précision). Décision d'implémentation prise et justifiée dans le code : découpage par paragraphe avec empaquetage glouton, taille cible 1500 caractères (300-400 tokens en français), chevauchement 200 caractères pour ne pas couper le contexte entre deux chunks consécutifs. Approximation par caractères plutôt que par le vrai tokenizer BGE-M3, pour ne pas devoir charger le modèle (2 Go) uniquement pour compter des tokens.

Listing 2: `src/indexeur_texte.py` – découpage par paragraphe, empaquetage glouton

```

TAILLE_CHUNK_CARACTERES = 1500
CHEVAUCHEMENT_CARACTERES = 200

@staticmethod
def _decouper_en_chunks(texte, taille_max=TAILLE_CHUNK_CARACTERES,
                        chevauchement=CHEVAUCHEMENT_CARACTERES):
    paragraphes = [p.strip() for p in texte.split("\n") if p.strip()]

```

```

chunks, courant = [], ""
for paragraphe in paragraphes:
    if len(paragraphe) > taille_max:
        if courant:
            chunks.append(courant); courant = ""
        pas = max(taille_max - chevauchement, 1)
        for i in range(0, len(paragraphe), pas):
            chunks.append(paragraphe[i : i + taille_max])
        continue
    candidat = f"{courant}\n{paragraphe}" if courant else paragraphe
    if len(candidat) <= taille_max:
        courant = candidat
    else:
        chunks.append(courant)
        amorce = courant[-chevauchement:] if chevauchement else ""
        courant = f"{amorce}\n{paragraphe}" if amorce else paragraphe
if courant:
    chunks.append(courant)
return chunks

```

Un paragraphe seul plus grand que `taille_max` (ex. tableau aplati en un bloc de texte) est tronqué à la dure plutôt qu'ignoré, pour ne perdre aucun contenu.

5.2 Embeddings (BGE-M3)

Le modèle retenu par l'ADR 0002 est BAAI/bge-m3 (multilingue, via `sentence-transformers`), confirmé comme choix de production dans les documents de conception. La fonction d'embedding est injectable au constructeur d'`IndexeurTexte`, pour permettre les tests sans télécharger le modèle (2 Go) dans l'environnement de développement sans accès réseau. **Point de vigilance important** : cette fonction factice n'est utilisée QUE dans les tests automatisés – le chemin de production (ci-dessous) charge et exécute le vrai modèle, validé réellement en section 8.

Listing 3: `src/indexeur_texte.py` – chargement paresseux du vrai modèle

```

def _embarquer(self, textes: list[str]) -> list[list[float]]:
    if self._fonction_embedding is not None:
        return self._fonction_embedding(textes) # utilise UNIQUEMENT dans les tests
    if self._modele is None:
        from sentence_transformers import SentenceTransformer
        self._modele = SentenceTransformer(NOM_MODELE_EMBEDDING) # "BAAI/bge-m3"
    return self._modele.encode(textes, normalize_embeddings=True).tolist()

```

6 Indexation hybride : Chroma (sémantique) + BM25 (lexical)

Chaque chunk est indexé à la fois dans ChromaDB (recherche par similarité d'embeddings, persistée sur disque) et dans un index BM25 (recherche lexicale par mots-clés, via `rank_bm25`, reconstruit à la demande faute de persistance native de cette bibliothèque). Les deux classements sont fusionnés par *Reciprocal Rank Fusion* (RRF, $k = 60$, valeur usuelle de la littérature – Cormack, Clarke & Buettcher, 2009) plutôt qu'une moyenne pondérée des scores : les scores denses (cosinus, bornés entre 0 et 1) et BM25 (non bornés) ne sont pas sur la même échelle, alors que RRF ne s'appuie que sur le *rang* de chaque résultat dans chaque liste.

Listing 4: `src/indexeur_texte.py` – fusion Reciprocal Rank Fusion

```

@staticmethod
def _fusionner_rrf(listes_classees, top_k, k_rrf=60):
    scores = {}
    for liste in listes_classees:
        for rang, id_ in enumerate(liste):
            scores[id_] = scores.get(id_, 0.0) + 1.0 / (k_rrf + rang + 1)
    classement = sorted(scores.keys(), key=lambda i: scores[i], reverse=True)
    return classement[:top_k]

```

Point technique vérifié expérimentalement pendant le développement : `collection.get(ids=[...])` de ChromaDB ne garantit PAS l'ordre des ids demandés en retour – la méthode `rechercher` ré-

ordonne donc explicitement les résultats selon le rang RRF plutôt que de faire confiance à l'ordre renvoyé par Chroma.

7 Persistance SQLite

Le module `src/base_donnees.py` relie les objets Python (`Document`, `Chunk`, `Indicateur`) aux tables de `db/schema.sql`, avec deux règles de dédoublement :

Table	Règle de dédoublement
<code>document</code>	Contrainte <code>url</code> UNIQUE en base : <code>insérer_document</code> renvoie l'id existant si l'URL est déjà connue, ne duplique jamais.
<code>indicateur</code>	Pas de contrainte UNIQUE dans le schéma ; upsert applicatif sur (nom, période, région, code_bds) côté code, nécessaire car le pré-remplissage BDS (ADR 0004) est conçu pour tourner à répétition (tâche nocturne) sans accumuler de doublons à chaque exécution.

L'embedding (liste de flottants) est sérialisé en BLOB via le module standard `array` (`float32`), sans dépendance supplémentaire :

Listing 5: `src/base_donnees.py` – serialisation de l'embedding

```
def _embedding_vers_blob(embedding: list[float]) -> bytes:
    return array.array("f", embedding).tobytes()

def blob_vers_embedding(blob: bytes) -> list[float]:
    tampon = array.array("f")
    tampon.frombytes(blob)
    return list(tampon)
```

8 Scripts d'orchestration

Script	Rôle
<code>scripts/indexer_documents.py</code>	Chaine complète : découverte (<code>data/listing_urls.py</code>) à Extracteur à insertion SQLite à <code>IndexeurTexte.indexer</code> . Les documents HTML sont explicitement ignorés (ADR 0003). Option <code>-limite N</code> ajoutée pour permettre un test rapide sur quelques documents avant un run complet.
<code>scripts/preremplir_indicateurs.py</code>	Preremplissage des indicateurs curés depuis l'API BDS (ADR 0004) : fait maintenant le vrai upsert en base (avant le Sprint 2, le script affichait seulement un résumé sans écrire en SQLite).
<code>scripts/inspecter_index.py</code>	Outil de vérification/démonstration : affiche un résumé lisible de la base (documents, chunks, indicateurs) et, avec <code>-recherche "question"</code> , exécute une vraie recherche hybride – pensé pour montrer un résultat concret à l'encadrante.

9 Tests automatisés

Philosophie suivie tout au long du projet : utiliser de vraies bibliothèques dans les tests (`chromadb`, `rank_bm25`, `sqlite3`, `python-docx`) plutôt que des simulacres, en ne substituant que la partie réellement coûteuse à exécuter en local (le modèle d'embedding, 2 Go, sans accès réseau dans l'environnement de développement).

Fichier de tests	Nombre	Couvre
test_indexeur_texte.py	15	Chunking (limites exactes, chevauchement, tronçonnage), fusion RRF, recherche hybride bout en bout (Chroma + BM25 réels)
test_base_donnees.py	9	Création des tables, dédoublonnage document/indicateur, round-trip de l'embedding
test_indexer_documents.py	4	Chaîne complète sur un vrai fichier .docx généré à la volée, filtrage HTML, non-duplication
Total du projet (tous sprints)	52	52/52 verts au 20 juillet 2026

10 Validation en conditions réelles

Ce sandbox de développement n'a pas d'accès réseau vers hcp.ma ni la capacité de télécharger le modèle BGE-M3 (2 Go) : toute validation réelle du pipeline complet a donc été effectuée par Ayman, sur sa propre machine, le 20 juillet 2026. Les blocs ci-dessous reconstituent fidèlement (texte identique) la sortie réelle de ces exécutions.

10.1 Premier test, 3 documents

```
PS> python -m scripts.indexer_documents -limite 3

[Scraper] piece jointe arabe ignoree (hors scope V1) : hcp.ma/file/248485/
[Scraper] piece jointe arabe ignoree (hors scope V1) : hcp.ma/file/245585/
html | ignore_html | 0 chunks | Les comptes regionaux. PIB et depenses de...
Warning: You are sending unauthenticated requests to the HF Hub.
(telechargement de BAAI/bge-m3 : modules.json, config.json, pytorch_model.bin
1.52GB, model.safetensors 1.52GB, tokenizer, sentencepiece.bpe.model ... )
pdf | ok | 40 chunks | Les comptes regionaux. PIB et depenses de...
html | ignore_html | 0 chunks | Les comptes nationaux provisoires 2025...
Total : 3 document(s) traite(s), 40 chunk(s) indexe(s).
```

Première exécution réelle de la chaîne complète : téléchargement effectif du modèle BGE-M3, scraping réel de hcp.ma, extraction, chunking, indexation. Les 2 pièces jointes arabes sont correctement écartées (filtre de langue), les 2 pages HTML sont correctement ignorées (ADR 0003), et 1 vrai document PDF est extrait, découpé en 40 chunks et indexé.

10.2 Vérification de la recherche hybride

```
PS> python -m scripts.inspector_index -recherche "produit interieur brut"

=== Resume de la base SQLite ===
Documents : 1      Chunks : 40      Indicateurs: 0

=== Extraits de chunks (3 premiers) ===
chunk #1 (position 0, embedding de dimension 1024) :
    "Table des matieres Contexte economique national 04..."

=== Recherche de demonstration : "produit interieur brut" ===
1. (document #1, position 29)
    "Produit interieur brut de la region 2,1 2,0 33125 29921 10,7..."
2. (document #1, position 27)
    "Produit interieur brut (aux prix courants) par branche d'activite..."
```

```
3. (document #1, position 0)
"Table des matieres Contexte economique national..."
```

Point de vérification clé : la dimension d’embedding affichée est **1024**, la signature exacte de BGE-M3 – preuve que le vrai modèle a tourné, pas la fonction factice utilisée dans les tests. Les 5 résultats retournés (3 montrés ici) sont tous effectivement pertinents sur le sujet demandé, bien classés.

10.3 Run élargi, 15 documents

```
PS> python -m scripts.indexer_documents -limite 15

pdf | ok | 40 chunks | Les comptes regionaux...
pdf | ok | 119 chunks | Les comptes nationaux provisoires 2025...
pdf | ok | 54 chunks | Les comptes nationaux des secteurs institutionnels...
pdf | ok | 73 chunks | Les provinces du Sud du Royaume...
[Scraper] piece jointe arabe ignoree (hors scope V1) : (x4)
pdf | ok | 49 chunks | Budget economique exploratoire 2027...
pdf | ok | 33 chunks | Point de conjoncture N. 50, Juillet 2026
pdf | ok | 87 chunks | Note de conjoncture N. 48, Avril 2026
Total : 15 document(s) traite(s), 455 chunk(s) indexe(s).
```

Le modèle BGE-M3, téléchargé et mis en cache lors du premier test, se charge ici quasi instantanément (pas de nouveau téléchargement). 8 documents PDF réels indexés sans erreur, 455 chunks au total, filtre de langue toujours actif (4 pièces jointes arabes supplémentaires écartées). Ce lot n’a couvert que la catégorie Économie (limite atteinte avant les listings Marché du travail/Population).

10.4 Run complet, sans limite : 151 documents, DOCX validé

Ayman a ensuite lancé le script sans `-limite`, jusqu’à son terme, sur les 15 pages listing (Économie, Marché du travail, Population & démographie). Exécution longue (gros volume, certaines publications comportant plusieurs pièces jointes – ex. le RGPH 2024 compte à lui seul 9 PDF distincts) mais sans aucune erreur bloquante du début à la fin.

```
PS> python -m scripts.indexer_documents (extrait de fin de run)

html | ignore_html | 0 chunks | Note sur les resultats de l'enquete nationale sur la migrati
docx | ok | 43 chunks | Note sur les resultats de l'enquete nationale sur la migrati
pdf | ok | 371 chunks | Note sur les resultats de l'enquete nationale sur la migrati
...
html | ignore_html | 0 chunks | Allocution de Monsieur le Haut-Commissaire au Plan a l'occas
docx | ok | 12 chunks | Allocution de Monsieur le Haut-Commissaire au Plan a l'occas
...
Total : 151 document(s) traite(s), 6999 chunk(s) indexe(s).
```

Résultat final : **151 documents indexés, 6999 chunks au total**. Point clé : **2 vrais fichiers .docx** rencontrés et indexés avec succès (43 et 12 chunks) – le support DOCX (ADR 0005, section 4) est désormais validé en conditions réelles, pas seulement par les tests avec fixture générée. Plusieurs cas limites réels ont été correctement gérés sans interrompre le run : liens annoncés comme PDF/DOCX mais dont le contenu téléchargé s’avère invalide (écartés via la vérification de signature binaire, Sprint 1), et filtre de langue arabe actif de bout en bout, sur les pages comme sur les pièces jointes individuelles.

11 Limites connues et travail restant

Point	État
Inventaire des pages listing Économie	Partiel : Indices des prix et production (IPC/IPPI/IPI/ICE), Secteurs d'activité et Sphère informelle restent à vérifier et ajouter à <code>data/listing_urls.py</code> .
Constructeur <code>Indicateurs.structure</code>	Non implémenté (repli PDF/XLSX pour les indicateurs hors catalogue BDS) : priorité volontairement basse, l'API BDS (ADR 0004) couvre déjà la majorité des indicateurs ciblés.
Piste <code>hcp.ma/downloads/?tag=X</code>	Découverte alternative trouvée (base de téléchargements filtrable par tag, liens directs) mais volontairement non exploitée – changement d'architecture plus lourd, à discuter dans un éventuel ADR ultérieur.

12 Transition vers le Sprint 3

Le Sprint 2 est désormais validé de bout en bout en conditions réelles, et pas seulement fonctionnellement complet côté code : chunking, embeddings BGE-M3 réels, indexation Chroma/BM25, persistance SQLite, et les trois formats de fichiers ciblés (PDF, XLSX, DOCX) tous confirmés sur de vraies publications hcp.ma, à l'échelle (151 documents, 6999 chunks). Le Sprint 3 peut s'ouvrir sur des bases solides : routage des questions, recherche structurée sur les indicateurs BDS, reranking, et génération de réponse.